LAPORAN ESSAI WEB APPLICATION DEVELOPMENT

Proyek Pengembangan Aplikasi Web E-Commerce

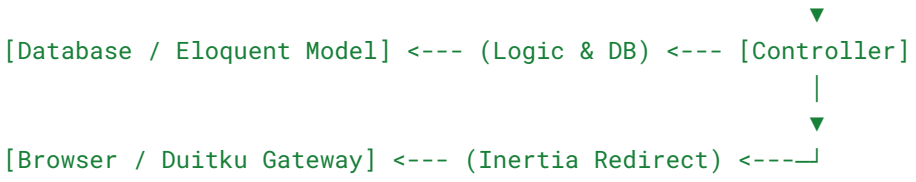
Nama Project	: Toko Online dengan Laravel, Inertia.js dan React.js
Nama	: Dimas Kurniawan
NIM	: 25120300016

---

1. Alur Kerja MVC (Request Life Cycle Fitur Penambahan/Checkout Data)

Siklus permintaan (*request life cycle*) dari pengguna hingga data berhasil disimpan di basis data menggunakan arsitektur MVC (Model-View-Controller) dengan kombinasi Inertia.js:

```
[Browser / React View] ---> (HTTPS Request) ---> [Laravel Router]
                                                        |
```



Tahap 1: Interaksi di View (Frontend React)

1. **User Action**: Pengguna menekan tombol **"BAYAR SEKARANG"** di halaman checkout (resources/js/Pages/Web/Checkouts/Index.jsx).
2. **State & Event Handler**: Elemen form memicu fungsi storeCheckout(). Fungsi ini mengumpulkan data state alamat, ongkir, serta payment_method (misalnya VA / VC).
3. **Inertia Request**: Inertia mengirimkan HTTP POST request berisi *payload JSON* ke rute /checkouts backend tanpa memuat ulang (*reload*) seluruh halaman.

Tahap 2: Routing & Control di Backend (Laravel Controller)

4. **Routing**: routes/web.php mengarahkan HTTP POST /checkouts ke method store() di CheckoutController.php.
5. **Validation & Security**:
    - Controller melakukan validasi parameter wajib (province_id, city_id, district_id, courier_cost, dll.).
    - Controller menghitung ulang harga dari database melalui Model Cart untuk mencegah manipulasi harga (*price tampering*).
6. **Integrasi Payment Gateway**:
    - Controller membuat *Signature MD5* Duitku dan menyusun payload transaksi.
    - Request dikirim via HTTP Client ke API Endpoint Duitku Sandbox.

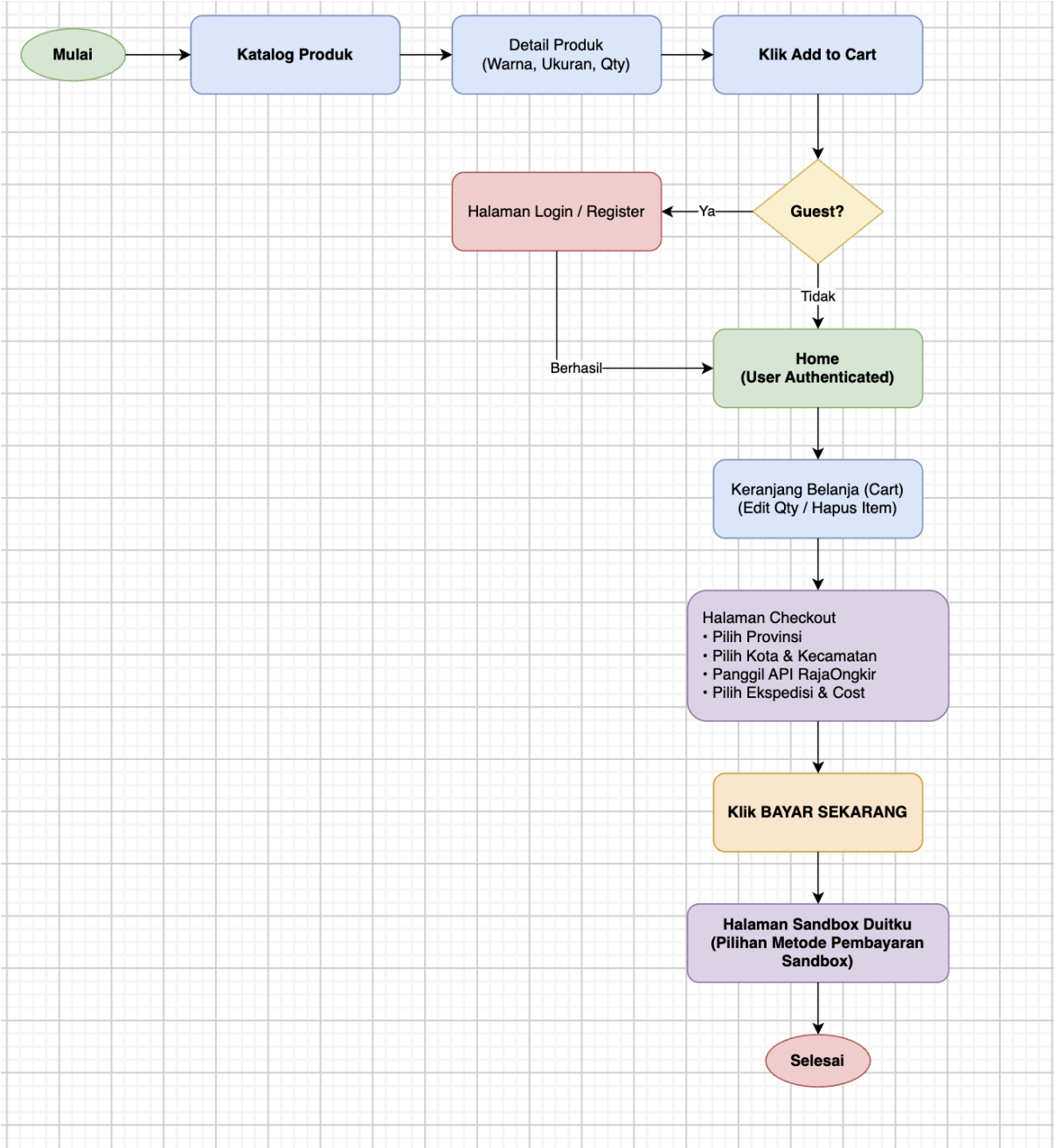
Tahap 3: Interaksi dengan Model & Database (Laravel Model)

7. **Database Transaction**:
    - Jika API Duitku merespons sukses (statusCode == '00'), Controller membuka transaksi basis data (DB::transaction).
    - Model Invoice menyimpan data transaksi baru (status UNPAID, detail pengiriman, dan paymentUrl dari Duitku).
    - Model Cart menghapus (*delete*) item keranjang belanja milik user tersebut.

Tahap 4: Respons Kembali ke View (Frontend Response)

8. **Inertia Location Redirect**: Controller mengembalikan respons Inertia::location($result['paymentUrl']).
9. **Browser Navigation**: Browser secara otomatis mengalihkan (*redirect*) halaman pengguna dari **2R STORE** menuju halaman transaksi Sandbox Duitku.

1.1 Alur Diagram Customer (Customer Journey Flow)

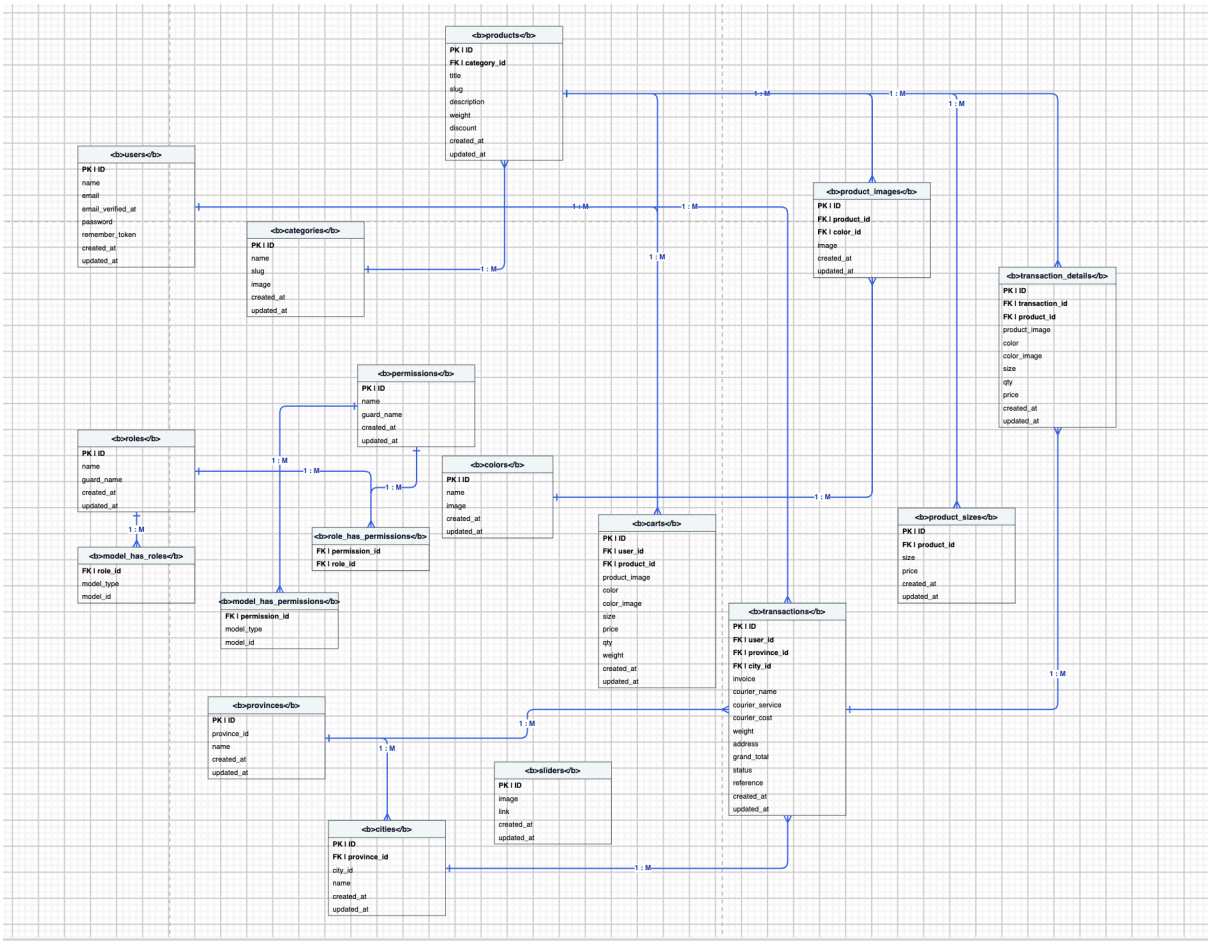


1. **Pemilihan Produk**:
    - Customer masuk ke halaman toko, memilih produk, lalu memilih variasi **Warna**, **Ukuran**, dan memasukkan **Jumlah (Qty)**.
2. **Autentikasi Pengguna (Guest Guard)**:
    - Saat tombol **"Add to Cart"** diklik, sistem mengecek status login. Jika belum login, customer otomatis diarah ke halaman **Login / Register**.
    - Setelah sukses mendaftar/masuk, customer kembali ke halaman **Home** dengan nama profil yang sudah terautentikasi (misal: *fitriani*).
3. **Manajemen Keranjang Belanja**:
    - Customer dapat membuka halaman keranjang untuk melakukan penyesuaian (mengubah jumlah pesanan atau menghapus produk).
4. **Proses Pengiriman (Integrasi RajaOngkir)**:

    - Customer melangkah ke halaman **Checkout**.
    - Memilih bertahap: **Provinsi** → **Kota** → **Kecamatan**.
    - Sistem secara otomatis memanggil **API RajaOngkir** (/checkouts/check-ongkir) untuk menampilkan daftar opsi ekspedisi (JNE, POS, J&T, Ninja) beserta tarif ongkos kirim real-time.
5. **Pembayaran Gateway (Sandbox Duitku)**:
    - Customer memilih layanan pengiriman, mengisi alamat lengkap, lalu menekan tombol **"BAYAR SEKARANG"**.
    - Sistem membuat invoice dan mengarahkan customer ke gateway **Sandbox Duitku**.
    - Pada mode Sandbox gratis (Free Sandbox Account), Duitku menyediakan channel testing universal (seperti *Virtual Account* Permata/CIMB/Maybank VA atau Credit Card VC) untuk menyelesaikan simulasi pembayaran transaksi.

2. Desain Basis Data (ERD & Relasi Tabel)

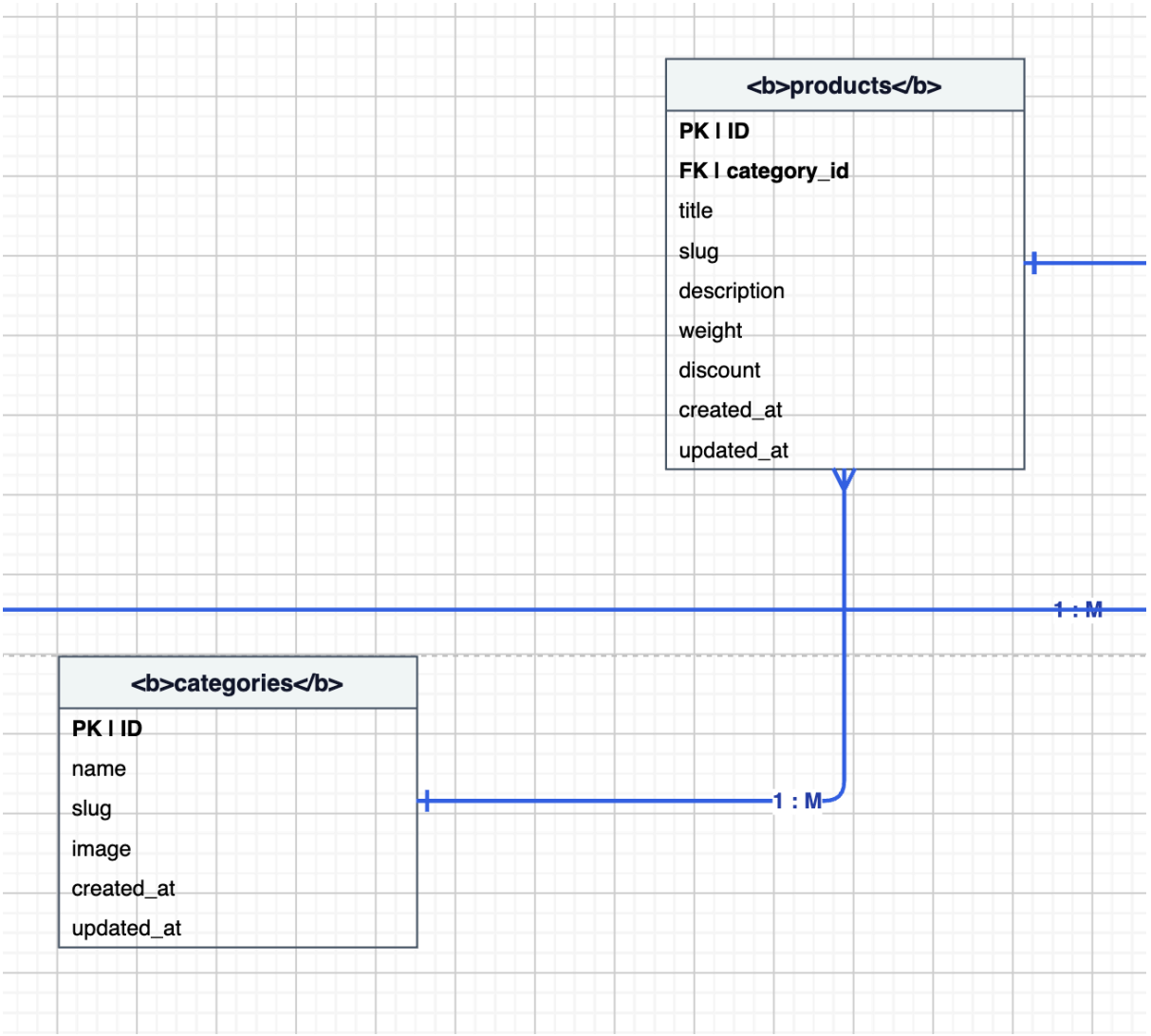
Tautan Diagram ERD: Diagram ERD Interactive Link



2.1 Pemilihan Tabel & Jenis Relasi

Dua contoh relasi utama yang diterapkan dalam sistem ini meliputi:

- **Relasi *One-to-Many* (1:N)**: Antara tabel **categories** dan **products**.



2.2 Alasan Pemilihan Relasi

- **Satu Kategori Memiliki Banyak Produk (*One-to-Many*)**:
    - **Alasan**: Sebuah produk (misal: *Jaket Parka*) hanya dikategorikan ke dalam 1 kategori spesifik (*Pakaian Pria*). Namun, 1 kategori *Pakaian Pria* dapat menampung banyak produk berbeda.

2.3 Implementasi di Tingkat Database (Foreign Key & Migration)

A. Relasi *One-to-Many* (categories → products)

Ditandai dengan menempatkan kolom *Foreign Key* category_id di dalam tabel anak (products).

- **Script Migration (database/migrations/2026_10_27_163234_create_products_table.php)**:
- Kode Schema Migration (PHP):

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    // Foreign Key merujuk ke id di tabel categories

$table->foreignId('category_id')->constrained('categories')->onDelete('cascade');
    $table->string('title');
    $table->text('description');
    $table->bigInteger('price');
    $table->integer('weight');
    $table->integer('stock');
    $table->timestamps();
});
```

2.4 Pemanggilan Data Relasional Menggunakan Eloquent ORM

A. Definisi Relasi pada Model Laravel

- **Model Product (app/Models/Product.php)**:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    // Relasi One-to-Many (Inverse): Product milik 1 Category
    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    // Relasi Many-to-Many: Product memiliki banyak Color
    public function colors()
    {
        return $this->belongsToMany(Color::class);
    }
}
```

- **Model Category (app/Models/Category.php)**:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
```

```php
class Category extends Model
{
    // Relasi One-to-Many: Category memiliki banyak Product
    public function products()
    {
        return $this->hasMany(Product::class);
    }
}
```

B. Cara Pemanggilan Data di Controller

Data relasional dipanggil secara efisien menggunakan teknik ***Eager Loading*** **(with())** untuk menghindari *N+1 Query Problem*:
Pemanggilan Eager Loading (PHP):

```php
// app/Http/Controllers/Apps/ProductController.php

use App\Models\Product;

public function index()
{
    // Mengambil data produk beserta data kategori dan warna terkait
    $products = Product::with(['category', 'colors'])
        ->latest()
        ->paginate(10);

    return inertia('Apps/Products/Index', [
        'products' => $products
    ]);
}
```

3. Keamanan Web (Security)

Serangan XSS terjadi ketika peretas menyisipkan skrip berbahaya (seperti JavaScript) melalui input form yang kemudian dieksekusi di browser pengguna lain. Dalam aplikasi ini, XSS dicegah melalui **dua lapis proteksi (Backend & Frontend)**:

- **Sisi Frontend (React.js Auto-Escaping):**
    - React secara default memperlakukan semua nilai variabel yang di-render di dalam kurung kurawal JSX ({ variable }) sebagai *plain text*, bukan sebagai kode HTML/JavaScript yang dapat dieksekusi.
    - Saat data produk atau input dari pengguna di-render di halaman React (misal: resources/js/Pages/Apps/Products/Index.jsx), string berbahaya seperti <script>alert('XSS')</script> secara otomatis di-escape menjadi entitas HTML aman (<script>...).

3.1 Pencegahan Serangan Cross-Site Scripting (XSS)

Diambil dari file komponen tampilan React (JSX), contohnya pada file catalog & checkout:

- **`resources/js/Pages/Web/Home/Index.jsx`**

```
// Penggunaan JSX kurung kurawal {product.title} dan {formatRupiah(product.price)}
<h6 className="card-title fw-bold text-dark text-truncate
mb-1">{product.title}</h6>
<span className="fw-bold text-success fs-6">{formatRupiah(product.price)}</span>
```

- **`resources/js/Pages/Web/Checkouts/Index.jsx`**

```
// Render variabel string aman di dalam elemen HTML
<textarea className="form-control" value={address} onChange={(e) =>
setAddress(e.target.value)}></textarea>
```

Penjelasan: Di seluruh file `.jsx` ini, semua variabel yang di-render menggunakan kurung kurawal `{ }` secara otomatis di-*escape* oleh React DOM menjadi string/entitas HTML biasa, bukan script executable.

- **Sisi Backend (Sanitasi Input & Safe Data Binding):**
  - Server Laravel memvalidasi dan memproses data input melalui Request object ($request->validate([...])) sebelum diproses lebih lanjut.
  - **Cookies & Session Protection**: Status autentikasi dan token sesi disimpan dalam cookie HTTP-Only dan SameSite (config/session.php). Hal ini memastikan bahwa meskipun skrip jahat berhasil berjalan, skrip tersebut **tidak dapat membaca token sesi atau cookie pengguna via document.cookie** di browser.

Sisi Backend (Sanitasi Input & Safe Data Binding)

Diambil dari file controller backend aplikasi Laravel:

- **`app/Http/Controllers/Web/CheckoutController.php`**

```
// Validasi input form sebelum diproses
$request->validate([
    'province_id'     => 'required',
    'city_id'         => 'required',
    'district_id'     => 'required',
    'courier_name'    => 'required',
    'courier_service' => 'required',
    'courier_cost'    => 'required|numeric',
    'grand_total'     => 'required|numeric',
    'address'         => 'required',
    'payment_method'  => 'required|string',
]);
```

- **Penjelasan**: Penggunaan `$request->validate()` dan pengolahan data via Eloquent ORM (seperti `Cart::where(...)` atau `Invoice::create(...)`) secara otomatis membersihkan (*sanitize*) serta menggunakan *prepared statements* sehingga mencegah injection baik SQL maupun XSS berbasis payload input.

Cookies & Session Protection (HttpOnly & SameSite)

Diambil dari file konfigurasi keamanan sesi dan environment Laravel:

- **`config/session.php`**

```
// Pengaturan default proteksi cookie sesi Laravel
'http_only' => env('SESSION_HTTP_ONLY', true),
'same_site' => env('SESSION_SAME_SITE', 'lax'),
'secure'    => env('SESSION_SECURE_COOKIE'),
```

- **`config/cors.php`** & **`bootstrap/app.php`** (Middleware `EncryptCookies` & `VerifyCsrfToken`)
  Penjelasan: Konfigurasi ini memastikan cookie session dan token autentikasi disimpan dengan flag `HttpOnly` sehingga variabel `document.cookie` tidak bisa dibaca oleh script JavaScript peretas di sisi client.

3.2 Pencegahan Serangan SQL Injection

SQL Injection terjadi ketika penyerang memasukkan kueri SQL berbahaya ke dalam kolom input form untuk memanipulasi, membaca, atau menghapus basis data server.

- **Penggunaan Eloquent ORM & Query Builder (PDO Parameter Binding):**
  - Seluruh transaksi data ke database tidak menggunakan kueri SQL mentah (*raw query string concatenation*), melainkan memanfaatkan **Eloquent ORM** dan **Query Builder** Laravel.
  - Contoh pada controller (app/Http/Controllers/Apps/ProductController.php):

```
// Input dimasukkan melalui PDO Prepared Statement
Product::create([
    'category_id' => $request->category_id,
    'title'       => $request->title,
    'description' => $request->description,
    'price'       => $request->price,
    'weight'      => $request->weight,
    'stock'       => $request->stock,
]);
```

  - **Mekanisme Kerja**: Laravel dan PDO (*PHP Data Objects*) secara otomatis menerapkan *parameter binding* (prepared statements). Nilai input dikirim terpisah dari struktur perintah SQL. Akibatnya, karakter khusus SQL seperti '

OR '1'='1 dianggap sebagai karakter string biasa, bukan sebagai perintah manipulasi SQL.

3.3 Ringkasan Lapisan Keamanan Form Input

Jenis Serangan	Lapisan Proteksi	Implementasi Kode / Script
XSS	JSX Auto-Escaping & HTTP-Only Cookies	resources/js/Pages/... & config/session.php
SQL Injection	Eloquent ORM / Prepared Statements	app/Http/Controllers/Apps/ProductController.php
CSRF	CSRF Token Validation Middleware	bootstrap/app.php & HandleInertiaRequests.php

4. Logika Autentikasi & Session

Proses autentikasi terjadi pada saat pengguna mengirimkan formulir login:

1. **Pengiriman Request**: Pengguna menginputkan *email* dan *password* dari form React (resources/js/Pages/Auth/Login.jsx) yang dikirim melalui method POST ke endpoint /login.
2. **Validasi Request Input**: Pada controller (app/Http/Controllers/Auth/LoginController.php), input terlebih dahulu divalidasi tipe data dan kelengkapannya menggunakan fungsi $request->validate().
3. **Pemeriksaan Kredensial & Hashing (Bcrypt)**:

    - Laravel menggunakan fasilitas Auth::attempt(['email' => $email, 'password' => $password]).
    - Server mencari data pengguna di tabel users berdasarkan email.
    - Kueri pencarian diproses menggunakan *parameter binding* (PDO) untuk mencegah **SQL Injection**.
    - Password yang diinput pengguna kemudian dicocokkan dengan hash yang ada di database menggunakan fungsi enkripsi **Bcrypt** (Hash::check()).
4. **Respon Login**: Jika kredensial cocok, Laravel membuat sesi baru. Jika gagal, controller mengembalikan *error response* beserta pesan kesalahan validasi ke antarmuka React.

4.1 Penyimpanan Informasi Session Secara Aman

Aplikasi tidak menggunakan *JSON Web Token* (JWT) yang ditaruh di *localStorage* browser (yang rentan terhadap pencurian via XSS), melainkan memanfaatkan **Stateful Session Cookies** berbasis server-side session:

- **Session ID Encryption**: Saat pengguna berhasil dikautentikasi, Laravel menggenerasi *Session ID* acak yang unik dan menyimpan data sesi pengguna secara terenkripsi di sisi server.
- **HTTP-Only & SameSite Cookies**: *Session ID* dikirimkan ke browser pengguna melalui cookie. Konfigurasi cookie (config/session.php) diatur ke:
    - http_only = true: Mencegah JavaScript di sisi klien (termasuk skrip XSS) mengakses isi cookie.
    - same_site = lax/strict: Melindungi aplikasi dari serangan *Cross-Site Request Forgery* (CSRF).
- **Regenerasi Session ID**: Setiap kali pengguna login, Laravel menjalankan $request->session()->regenerate() untuk mencegah serangan *Session Fixation*.

4.2 Proteksi Hak Akses (Unauthorized User)

Untuk memastikan halaman terproteksi tidak dapat diakses oleh pengguna yang belum terautentikasi atau tidak berhak, diterapkan mekanisme proteksi berlapis:

A. Proteksi Sisi Server (Laravel Middleware)

- **Penerapan Middleware auth**: Pada file rute backend (routes/web.php), halaman-halaman sensitif (seperti dashboard admin /apps/... atau halaman checkout /checkout) dikelompokkan dalam grup yang dilindungi oleh middleware auth:

- **Pencegahan Akses Langsung**: Ketika pengguna *unauthorized* mencoba mengakses URL terproteksi secara langsung melalui *address bar* browser, middleware auth memintercept permintaan tersebut. Karena tidak menemukan cookie sesi yang valid, server memblokir akses dan mengarahkan (*redirect*) pengguna secara otomatis ke halaman /login dengan status HTTP 401 Unauthorized atau 302 Found.

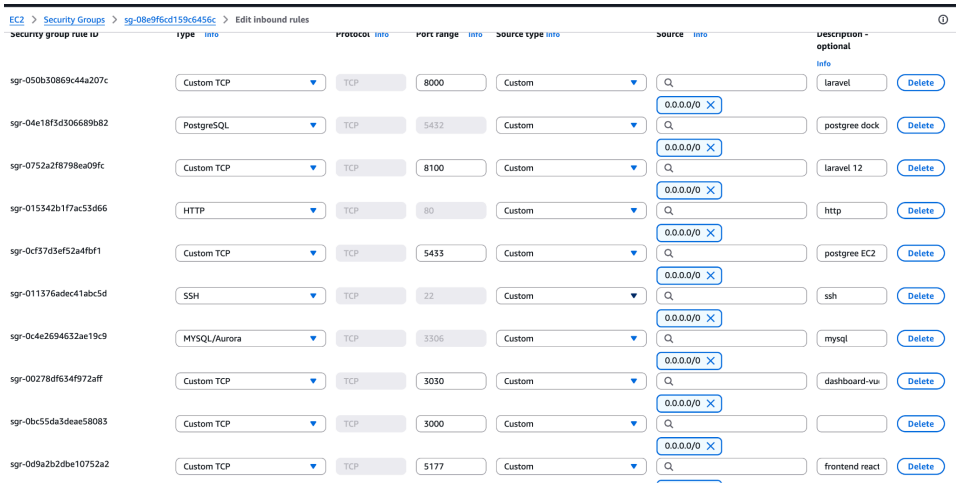
B. Operasional Sharing State via Inertia.js

- **Middleware HandleInertiaRequests**: File app/Http/Middleware/HandleInertiaRequests.php bertugas membagikan status autentikasi pengguna (auth.user) ke seluruh komponen React secara terenkripsi dan aman. Jika pengguna belum login, properti auth.user bernilai null sehingga komponen UI frontend dapat menyesuaikan tampilan secara otomatis.

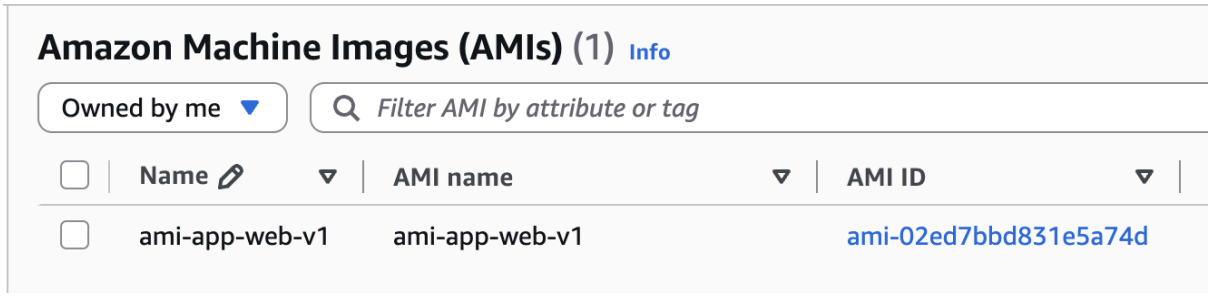
5. Manajemen Environment & Deployment

Penggunaan AWS EC2 mengubah beberapa tahapan infrastruktur dan pengelolaan server:

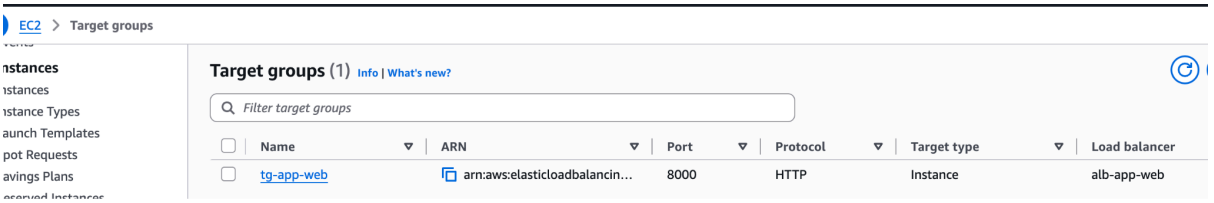
1. **Provisioning Instans EC2**:
    - Membuat instans EC2 baru (misal: *Amazon Linux 2023*) melalui AWS Management Console / CLI.
    - Mengonfigurasi **Security Group** (Firewall EC2) untuk membuka port akses:
        - Port `22` (SSH) – Terbatas hanya untuk IP pengembang.
        - Port `8000` (HTTP) & `443` (HTTPS) – Terbuka untuk umum (`0.0.0.0/0`).
        - Port 5177 dan 3306



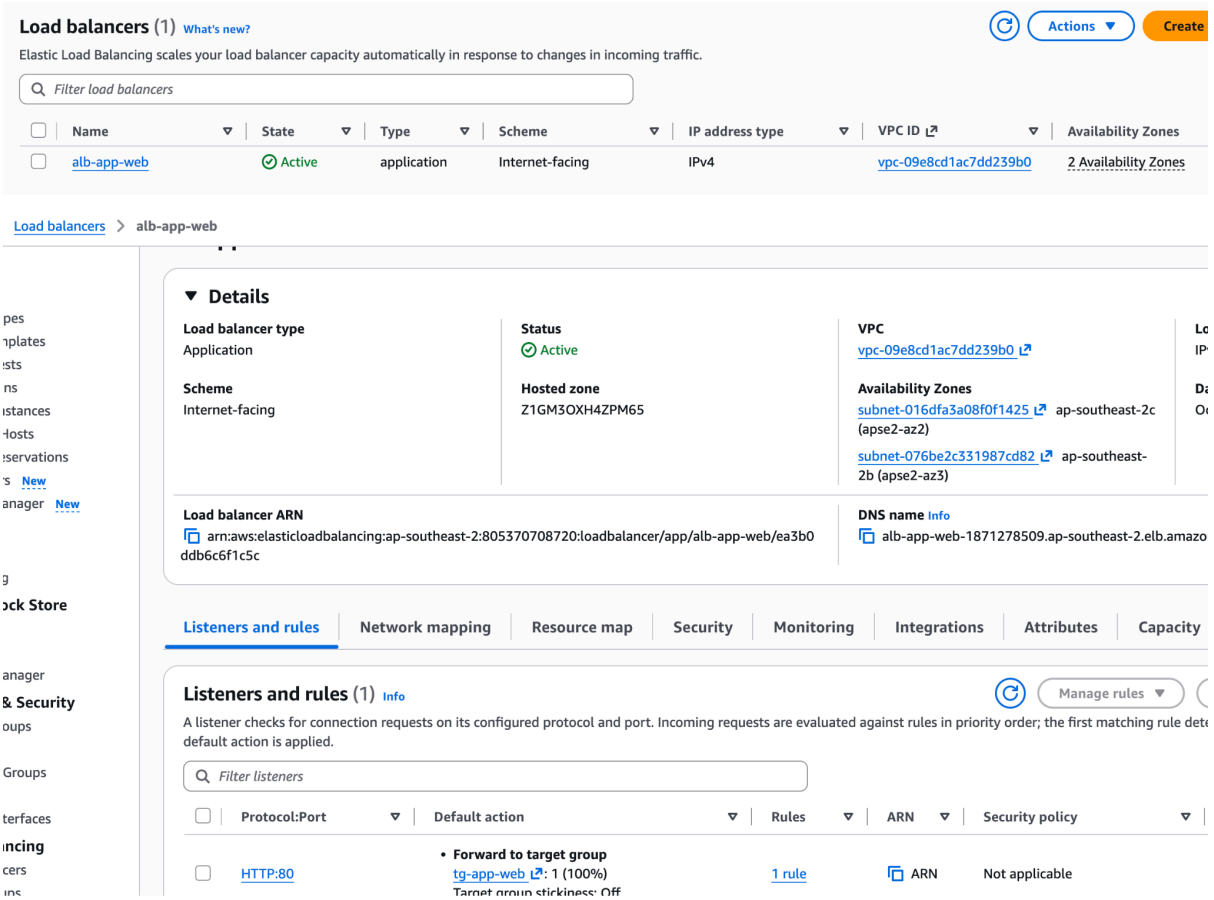
    - Menyiapkan Kunci Privat SSH (`.pem` / `.ppk`) untuk akses masuk ke server EC2.
2. create AMI Image



3. Create Target Group



4. Create Load balancing



5. **Koneksi & Instalasi Runtime**:
    - Masuk ke EC2 via SSH:

```
ssh -i "your-key.pem" ec2-user@ec2-xx-xx-xx-xx.compute-1.amazonaws.com
```

Buat service systemd

Buka file service:

```
sudo nano /etc/systemd/system/laravel.service
```

Masukkan konfigurasi berikut. Sesuaikan `WorkingDirectory` dengan lokasi project Laravel kamu.

```
[Unit]
Description=Laravel Artisan Development Server
After=network.target
```

```
[Service]
Type=simple
User=ec2-user
WorkingDirectory=/home/ec2-user/docker_web
ExecStart=/usr/bin/php artisan serve --host=0.0.0.0 --port=8000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Simpan di Nano: `Ctrl + O`, tekan `Enter`, lalu `Ctrl + X`.

Langkah 4 — Aktifkan dan jalankan otomatis

Jalankan command berikut:

```
sudo systemctl daemon-reload
sudo systemctl enable --now laravel
```

Perintah tersebut akan langsung menjalankan Laravel dan mengaktifkan service agar otomatis berjalan saat EC2 boot.

Langkah 5 — Cek status

```
sudo systemctl status laravel
```

Jika berhasil, status akan menunjukkan:
Active: active (running)

Langkah 6 — Melihat log dan mengelola service

Kebutuhan	Command
Melihat log langsung	`sudo journalctl -u laravel -f`
Restart Laravel	`sudo systemctl restart laravel`
Stop Laravel	`sudo systemctl stop laravel`
Start Laravel	`sudo systemctl start laravel`
Cek port 8000	`sudo ss -lntp | grep :8000`
Nonaktifkan auto-start	`sudo systemctl disable laravel`

5.1 Konfigurasi Akses Browser & Network Security

Pastikan Security Group EC2 mengizinkan inbound TCP port `8000` dari IP kamu. Hindari membuka port ini ke seluruh internet untuk aplikasi yang belum diamankan.

Akses menggunakan:
`http://32.236.168.110:8000/`
Catatan: `php artisan serve` merupakan development server Laravel, jadi untuk production sebaiknya gunakan Nginx atau Apache dengan PHP-FPM.

6. **Cloning & Konfigurasi Aplikasi**:
    - Mengkloning *repository* dari Git ke direktori server EC2.
    - Menginstal pustaka backend & frontend (`composer install --no-dev`, `npm install && npm run build`).
    - Mengonfigurasi file `.env` produksi di server.
7. **Koneksi Elastic IP & Domain / SSL**:
    - Mengasosiasikan **Elastic IP** (IP Publik Statis AWS) ke instans EC2 agar IP tidak berubah saat instans di-*restart*.
    - Mengarahkan DNS Domain ke Elastic IP dan mengonfigurasi SSL (Certbot / AWS Certificate Manager via AWS ALB jika menggunakan Load Balancer).

Link Repository: https://github.com/dimask016/project_uas_eligible

Akses Web menggunakan: ec2-32-236-168-110.ap-southeast-2.compute.amazonaws.com:8000

atau menggunakan Load balancing app nya :

alb-app-web-1871278509.ap-southeast-2.elb.amazonaws.com

DB : ec2-13-210-247-47.ap-southeast-2.compute.amazonaws.com

Struktur Folder:

```
geek-store/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Account/                        # [C] Controllers Khusus Admin &
Customer Dashboard
│   │   │   │   ├── CategoryController.php      # Resource controller kategori
(Admin)
│   │   │   │   ├── ColorController.php         # Resource controller warna
(Admin)
│   │   │   │   ├── DashboardController.php     # Single action controller
(__invoke) ringkasan transaksi
│   │   │   │   ├── PermissionController.php    # Management permission (Spatie)
│   │   │   │   ├── ProductController.php       # Resource controller produk
(Admin)
```

```
|   |   |   |   ├── RoleController.php            # Management role (Spatie)
|   |   |   |   ├── SliderController.php          # Resource controller slider
banner
|   |   |   |   ├── TransactionController.php     # Filter & tampilkan transaksi
(Admin & Customer)
|   |   |   |   └── UserController.php            # Management user/admin
|   |   |   |
|   |   |   ├── Auth/                             # [C] Controllers Autentikasi
|   |   |   |   ├── LoginController.php           # Login & session handling
|   |   |   |   └── RegisterController.php        # Registrasi customer baru
|   |   |   |
|   |   |   ├── Web/                              # [C] Controllers Web Public
Storefront
|   |   |   |   ├── CartController.php            # Keranjang belanja (CRUD Cart)
|   |   |   |   ├── CategoryController.php        # Katalog produk berdasarkan
kategori
|   |   |   |   ├── CheckoutController.php        # Checkout, Duitku Gateway, &
RajaOngkir API
|   |   |   |   ├── HomeController.php            # Landing page toko utama
|   |   |   |   ├── ProductController.php         # Halaman detail & daftar produk
|   |   |   |   └── SearchController.php          # Pencarian produk
|   |   |   |
|   |   |   └── Controller.php                    # Base Controller Laravel
|   |   |
|   |   └── Middleware/                           # Layer Proteksi & Keamanan
|   |       ├── HandleInertiaRequests.php         # Share global props ke React
(Auth user, Flash message)
|   |       └── IsAdmin.php                       # Custom middleware proteksi
admin
|   |
|   └── Models/                                   # [M] Database Models (Eloquent)
|       ├── Cart.php
|       ├── Category.php
|       ├── Color.php
|       ├── Product.php
|       ├── Slider.php
|       ├── Transaction.php                       # Model utama Invoice / Transaksi
Duitku
|       └── User.php                              # Model User dengan trait
HasRoles (Spatie)
|
├── bootstrap/
|   └── app.php                                   # Registrasi Middleware, CSRF
Exceptions, & App Routing
|
├── config/                                       # Konfigurasi Sistem &
Third-Party Services
|   ├── app.php                                   # App Name, Timezone, Locale, &
Providers
|   ├── auth.php                                  # Authentication Guards &
Password Reset
```

```
|   ├── cache.php                          # Caching Drivers (Redis/File)
|   ├── database.php                       # Koneksi Database MySQL /
MariaDB
|   ├── duitku.php / duitKu.php            # Credential & Endpoint Duitku
Sandbox / Production
|   ├── filesystems.php                    # Storage Disks (Public, Local,
S3)
|   ├── logging.php                        # Logging System Channels
|   ├── mail.php                           # Mailer Config
(SMTP/Notification)
|   ├── permission.php                     # Konfigurasi Spatie
Laravel-Permission
|   ├── queue.php                          # Background Queue Workers
Configuration
|   ├── rajaongkir.php                     # API Key & Account Type
(Starter/Pro) RajaOngkir
|   ├── services.php                       # Third-Party Credentials
Configuration
|   └── session.php                        # Session Drivers, HttpOnly, &
Cookie Protection
|
├── database/
|   ├── migrations/                        # Skema tabel database (Users,
Transactions, Carts, etc.)
|   └── seeders/                           # Seeder role 'admin' &
'customer'
|
├── resources/
|   └── js/                                # [V] Views - Single Page
Application via React.js
|       ├── Components/                    # Reusable UI Component (Header,
Navbar, Footer)
|       ├── Layouts/                       # Layout Utama
|       |   ├── Account.jsx                # Layout Dashboard Admin &
Customer (Sidebar Navigation)
|       |   └── Web.jsx                    # Layout Public Storefront
|       ├── Pages/                         # Inertia Page Components
|       |   ├── Account/                   # Halaman Dashboard
|       |   |   ├── Categories/            # View CRUD Kategori
|       |   |   ├── Colors/                # View CRUD Warna
|       |   |   ├── Dashboard/
|       |   |   |   └── Index.jsx          # Dashboard Admin (Summary Cards
& Chart Transaksi)
|       |   |   ├── Products/              # View CRUD Produk
|       |   |   ├── Roles/                 # View Management Role
|       |   |   ├── Sliders/               # View CRUD Slider
|       |   |   ├── Transactions/
|       |   |   |   ├── Index.jsx          # View Daftar Transaksi (Tab
Filter Status & Search)
|       |   |   |   └── Show.jsx           # Detail Transaksi & Status
Pembayaran
```

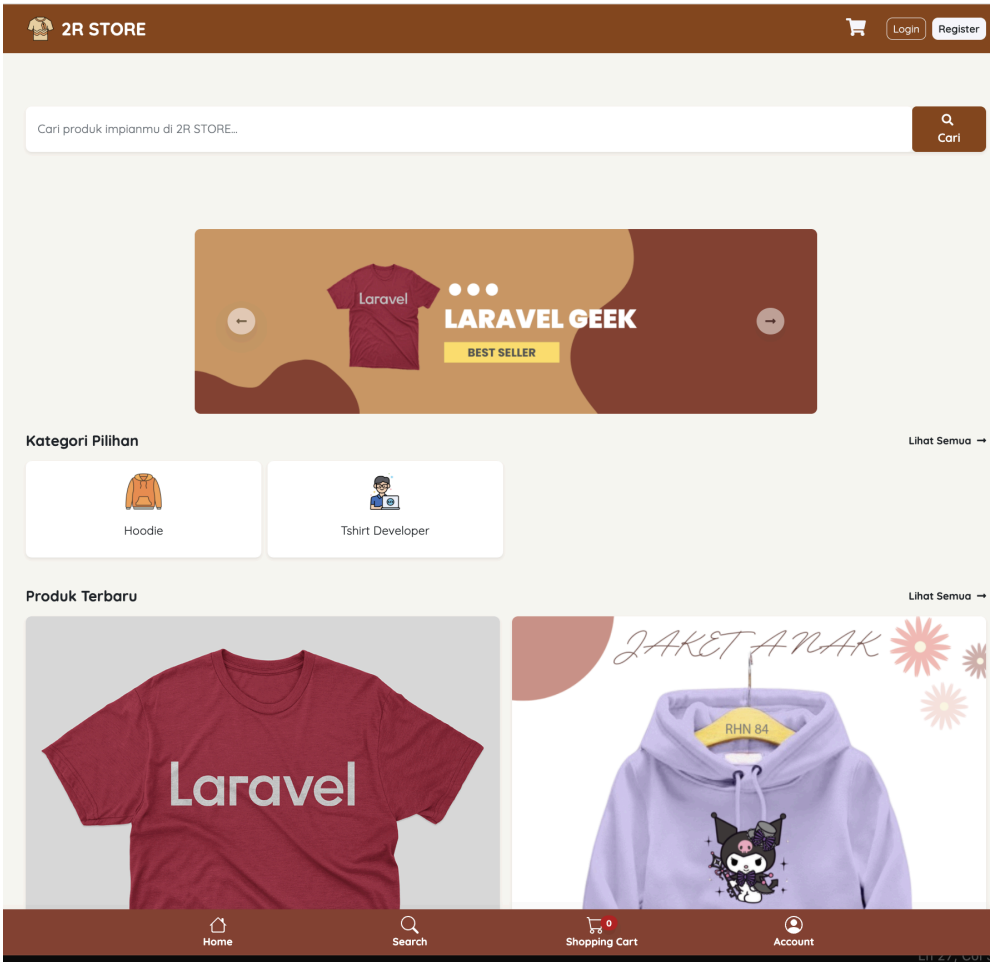
```
|       |   |   └── Users/                      # View Management User
|       |   |
|       |   ├── Auth/                           # Halaman Login & Register
|       |   |   ├── Login.jsx
|       |   |   └── Register.jsx
|       |   |
|       |   └── Web/                            # Halaman Public Storefront
|       |       ├── Carts/
|       |       |   └── Index.jsx               # Keranjang Belanja
|       |       ├── Checkouts/
|       |       |   └── Index.jsx               # Form Checkout (RajaOngkir &
Duitku Payment Channel)
|       |       ├── Home/
|       |       |   └── Index.jsx               # Landing Page Produk
|       |       └── Products/
|       |           ├── Index.jsx
|       |           └── Show.jsx                # Detail Produk
|       |
|       └── Utils/
|           └── FormatPrice.js                  # Helper format mata uang
(Rupiah)
|
└── routes/
    ├── console.php
    └── web.php                                 # [R] Routing Utama (Role-Based
Middleware Protection)
```

5.2 List Fitur Aplikasi

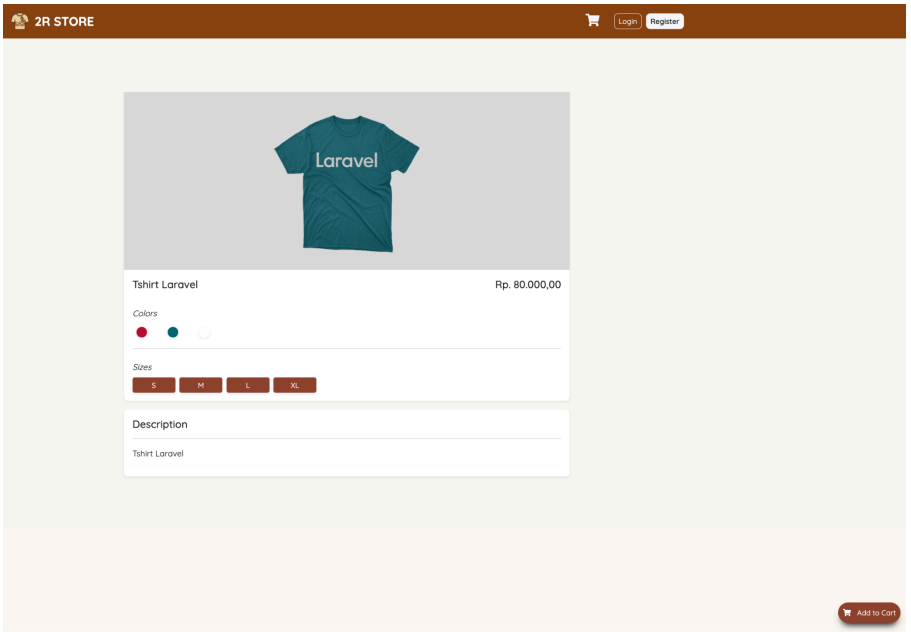
Berikut adalah daftar fitur lengkap aplikasi **2R STORE** yang terbagi untuk sisi **Customer (Web Public)** dan **Dashboard Admin**:

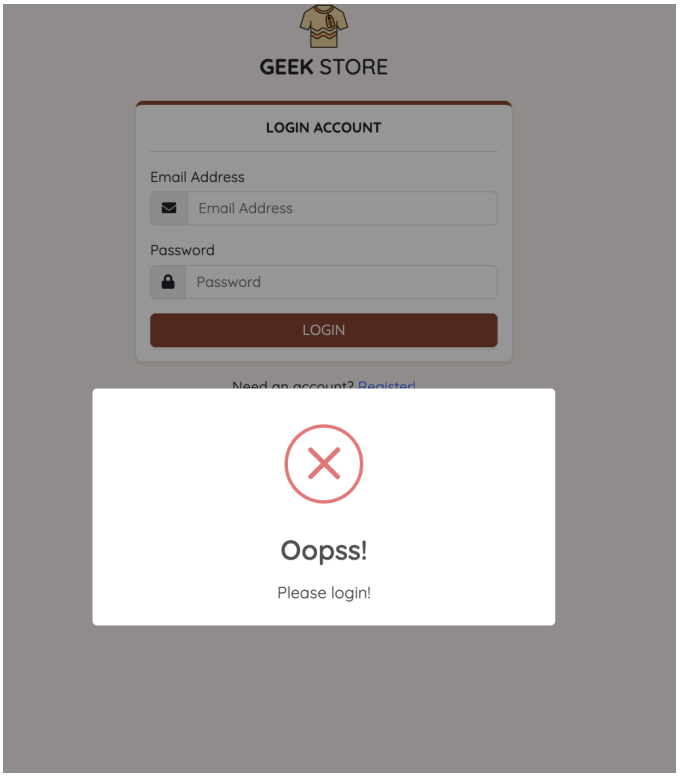
1. Fitur Sisi Customer (Web Public Storefront)

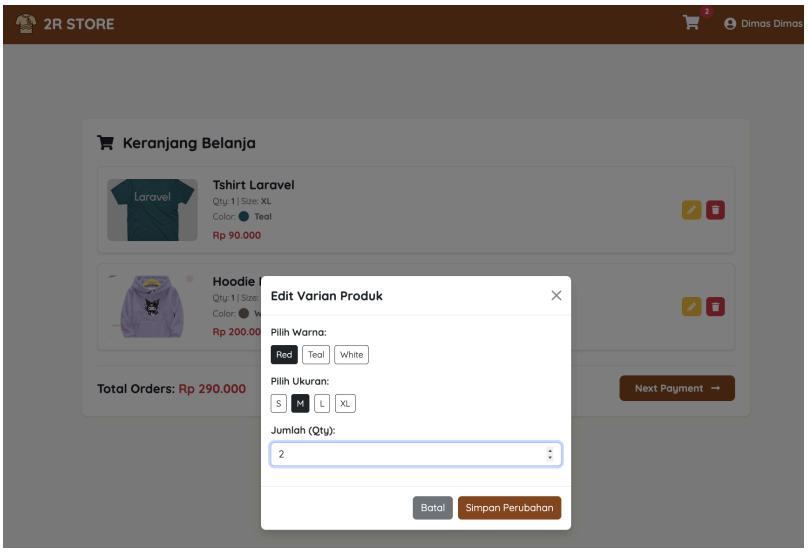
- **E-Katalog & Pencarian Produk**:
    - Menampilkan katalog produk terbaru, rekomendasi, dan slider banner di halaman utama (`Home`).
    - Filter produk berdasarkan Kategori (`Categories`) dan pencarian kata kunci (`Search`).
    - Detail produk lengkap beserta harga, varian (warna), deskripsi, dan stok.

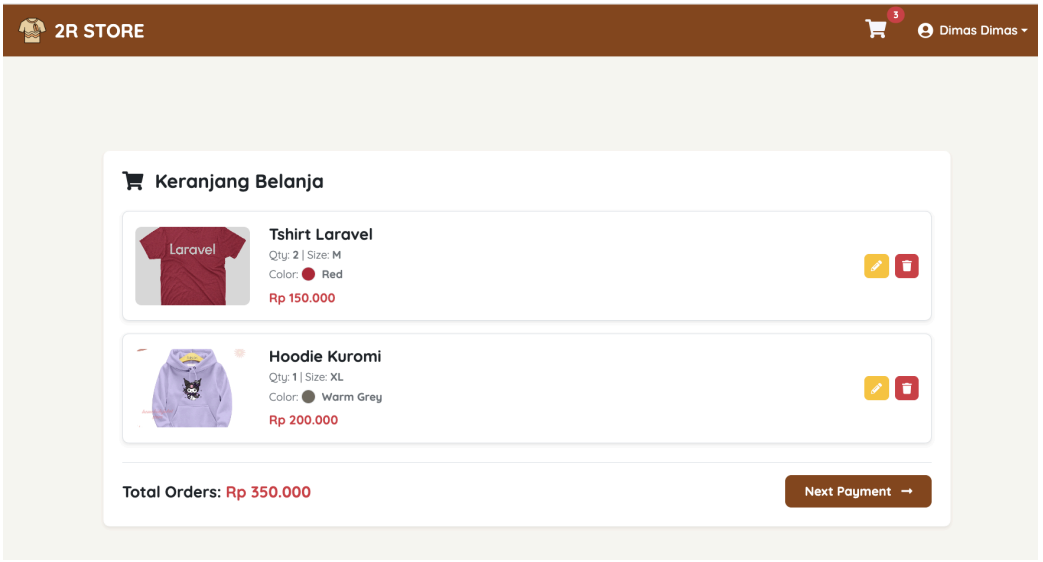


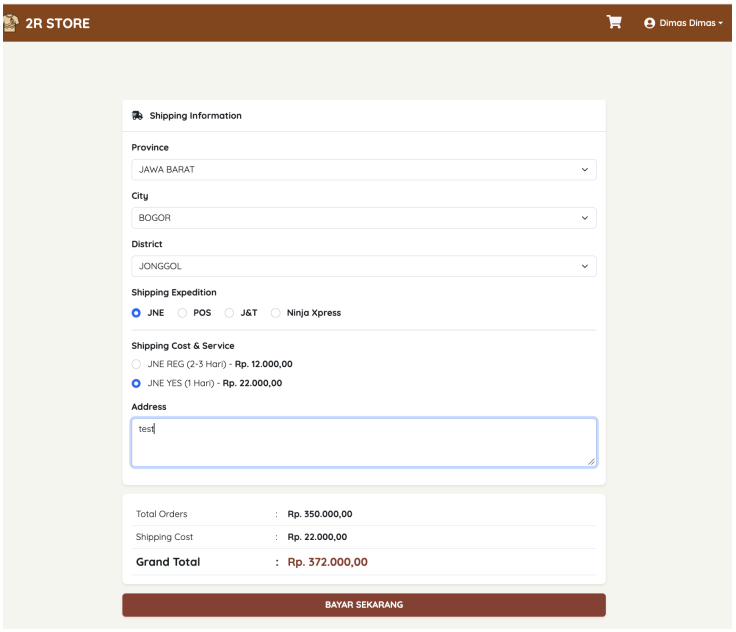
- **Keranjang Belanja (Cart Management)**:
    - Tambah produk ke keranjang belanja (mendukung guest & authenticated user).
    - Update kuantitas item dan hapus item dari keranjang.



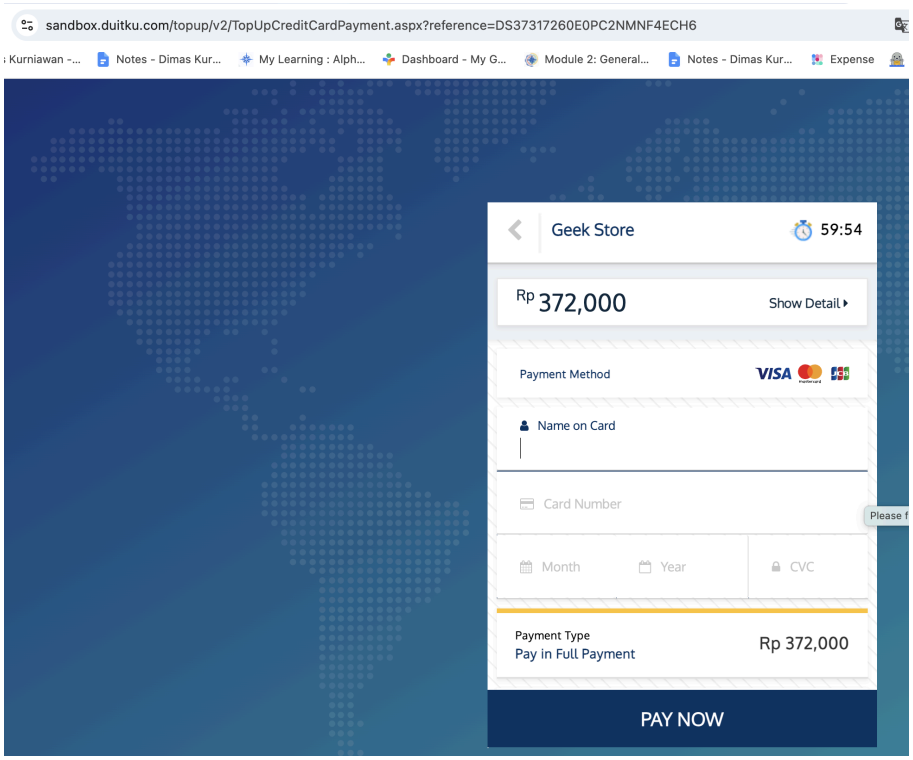

GEEK STORE
LOGIN ACCOUNT
Email Address
Email Address
Password
Password
LOGIN
Need an account? Register!
Oopss!
Please login!


2R STORE
Dimas Dimas
Keranjang Belanja
Tshirt Laravel
Qty: 1 | Size: XL
Color: Teal
Rp 90.000
Hoodie
Rp 200.00
Total Orders: Rp 290.000
Next Payment
Edit Varian Produk
Pilih Warna:
Red
Teal
White
Pilih Ukuran:
S
M
L
XL
Jumlah (Qty):
2
Batal
Simpan Perubahan



- **Proses Checkout & Hitung Ongkir Otomatis**:
  - Integrasi **API RajaOngkir** untuk memilih Kota, Kecamatan, dan kalkulasi biaya pengiriman secara real-time.
  - 

- **Multi-Payment Gateway Integration**:
  - Integrasi **Duitku Sandbox** untuk pembayaran instan (Virtual Account/VA, Credit Card, dll.).
  - Generasi otomatis invoice dan kode bayar (`snap_token`).



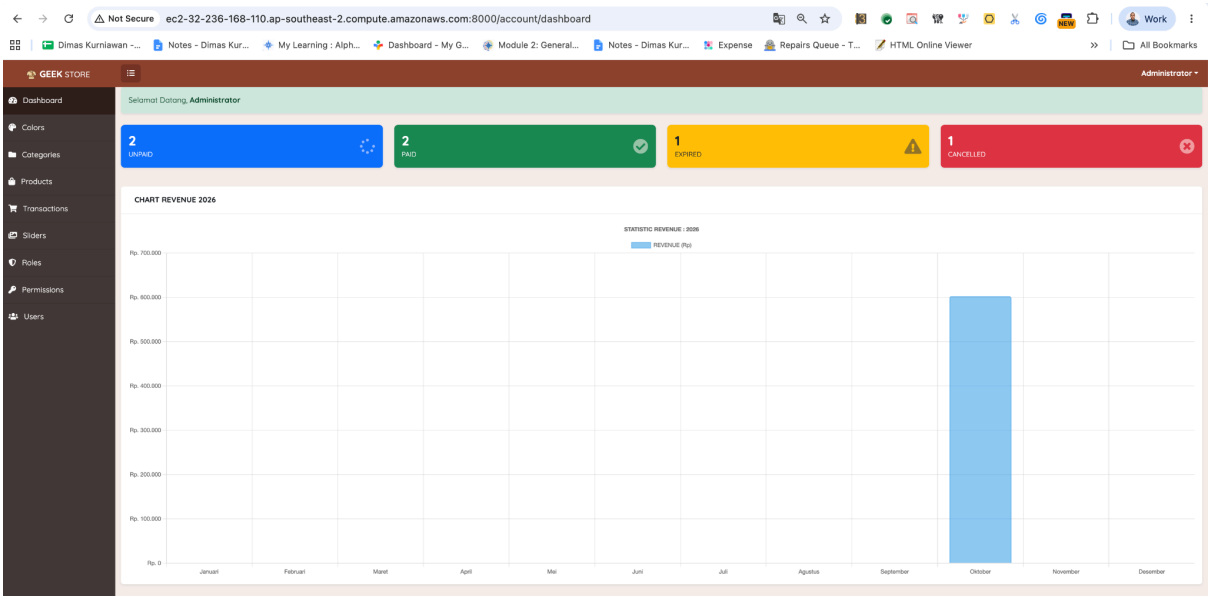
- **Manajemen Akun & Transaksi Customer**:
    - Autentikasi (Registrasi Akun Baru, Login, & Logout).
    - Halaman **"Transaksi Saya"** (`/account/transactions`) untuk memantau status pesanan milik sendiri (UNPAID, PAID, EXPIRED, CANCELLED).
    - Detail invoice dan instruksi pembayaran.

2. Fitur Sisi Dashboard Admin

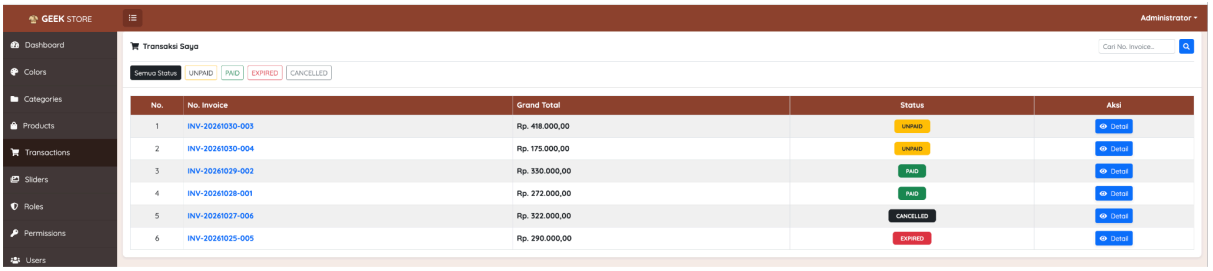
http://ec2-32-236-168-110.ap-southeast-2.compute.amazonaws.com:8000/login

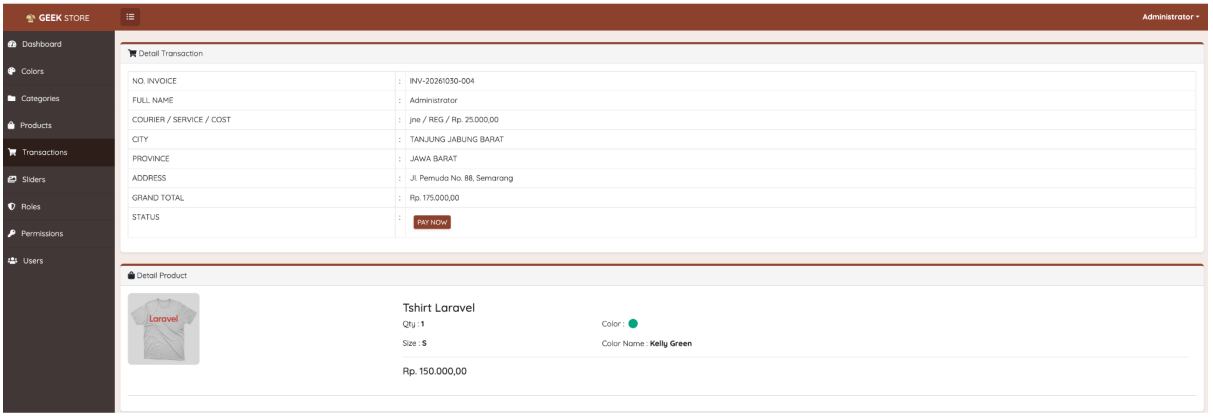
username : admin@gmail.com
password : password

- **Autentikasi & Proteksi Akses (RBAC)**:
    - Restriksi rute dashboard (`/account/dashboard`) khusus role **Administrator** menggunakan **Spatie Laravel-Permission** & Middleware.
    - Pengalihan (redirect) otomatis untuk user non-admin / customer yang mencoba mengakses area sensitif.

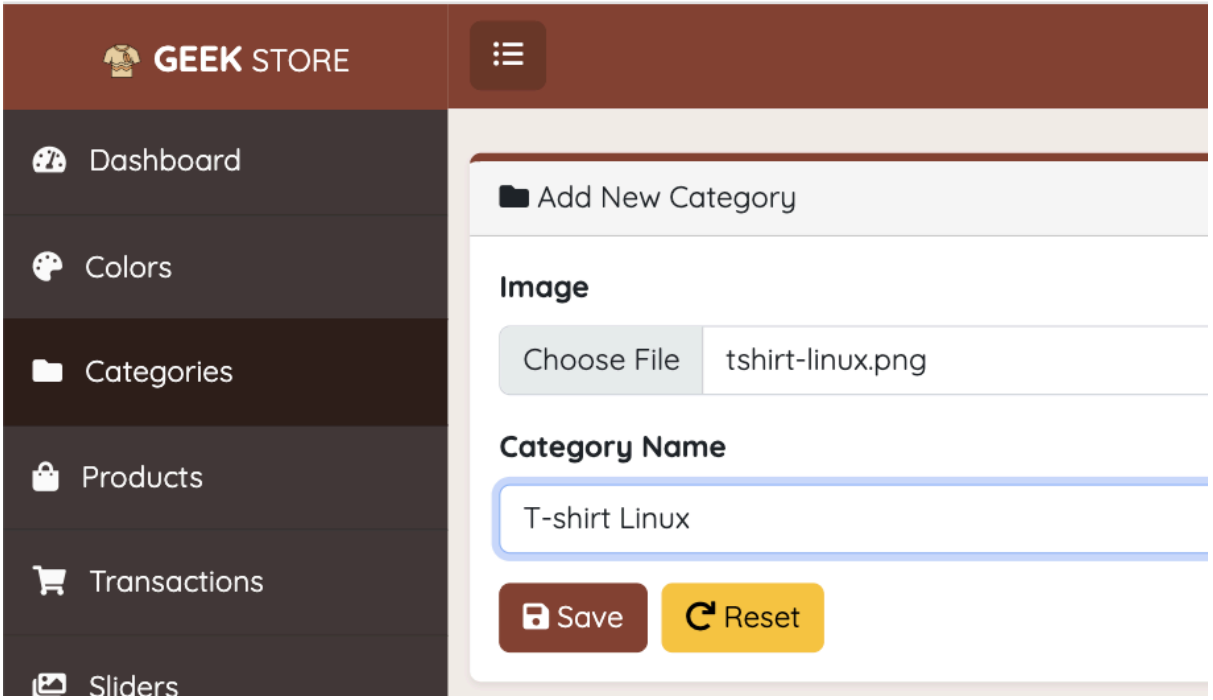


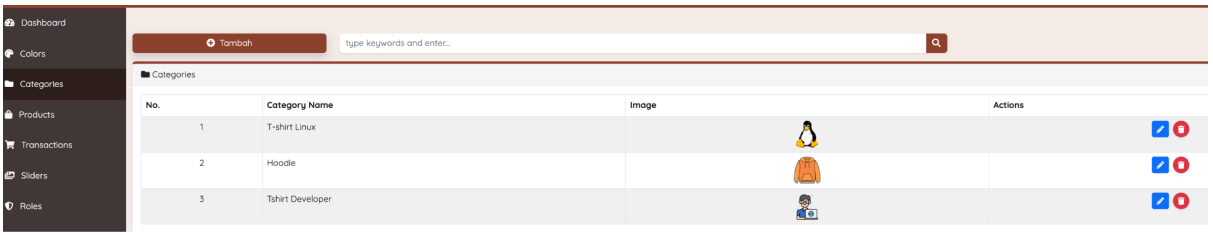
- **Analistik & Dashboard Ringkasan**:
  - Ringkasan statistik jumlah transaksi (UNPAID, PAID, EXPIRED, CANCELLED).
  - Grafik/Chart laporan pendapatan bulanan transaksi berstatus `PAID`.
- **Manajemen Transaksi Customer (Order Management)**:
  - Transparansi melihat seluruh transaksi dari semua customer di toko.
  - Tab filter status transaksi cepat (Semua Status, UNPAID, PAID, EXPIRED, CANCELLED).
  - Fitur pencarian transaksi berdasarkan Nomor Invoice.
  - Halaman detail transaksi customer (item yang dibeli, alamat pengiriman, dan status pembayaran).



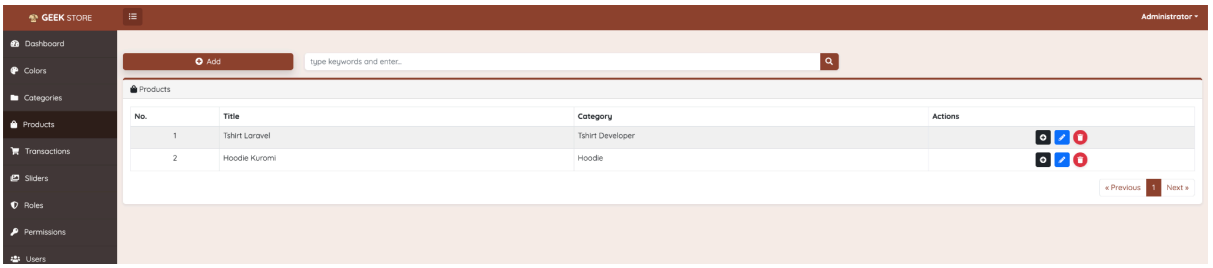


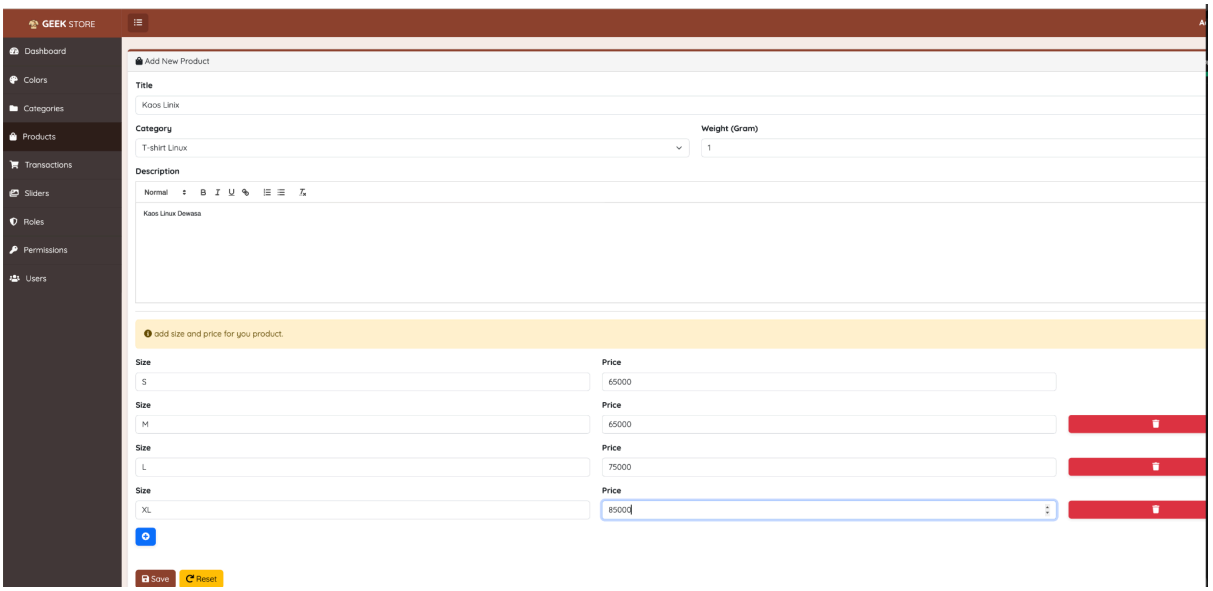
- **Manajemen Produk & Katalog (CRUD)**:
  - **Categories Management**: Tambah, ubah, hapus, dan atur kategori produk.



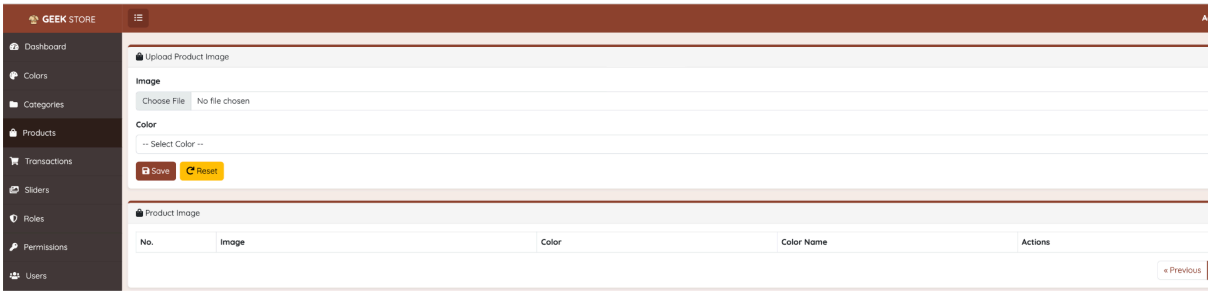


- **Products Management**: Kelola data produk (nama, deskripsi, harga, stok, warna, dan gambar produk).

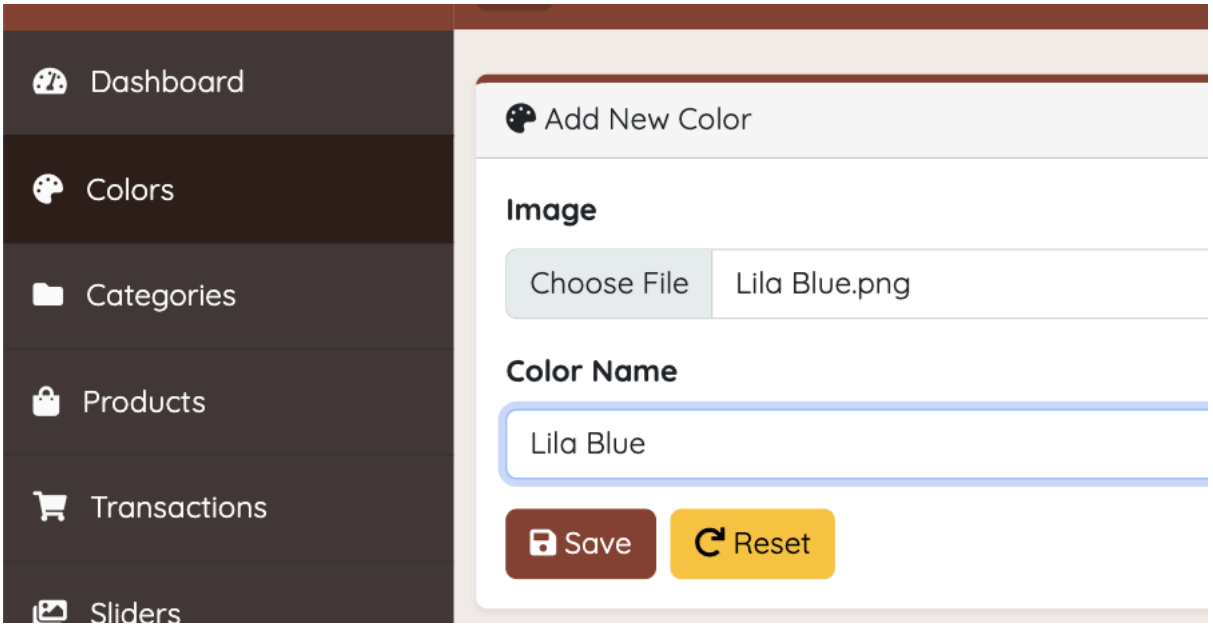


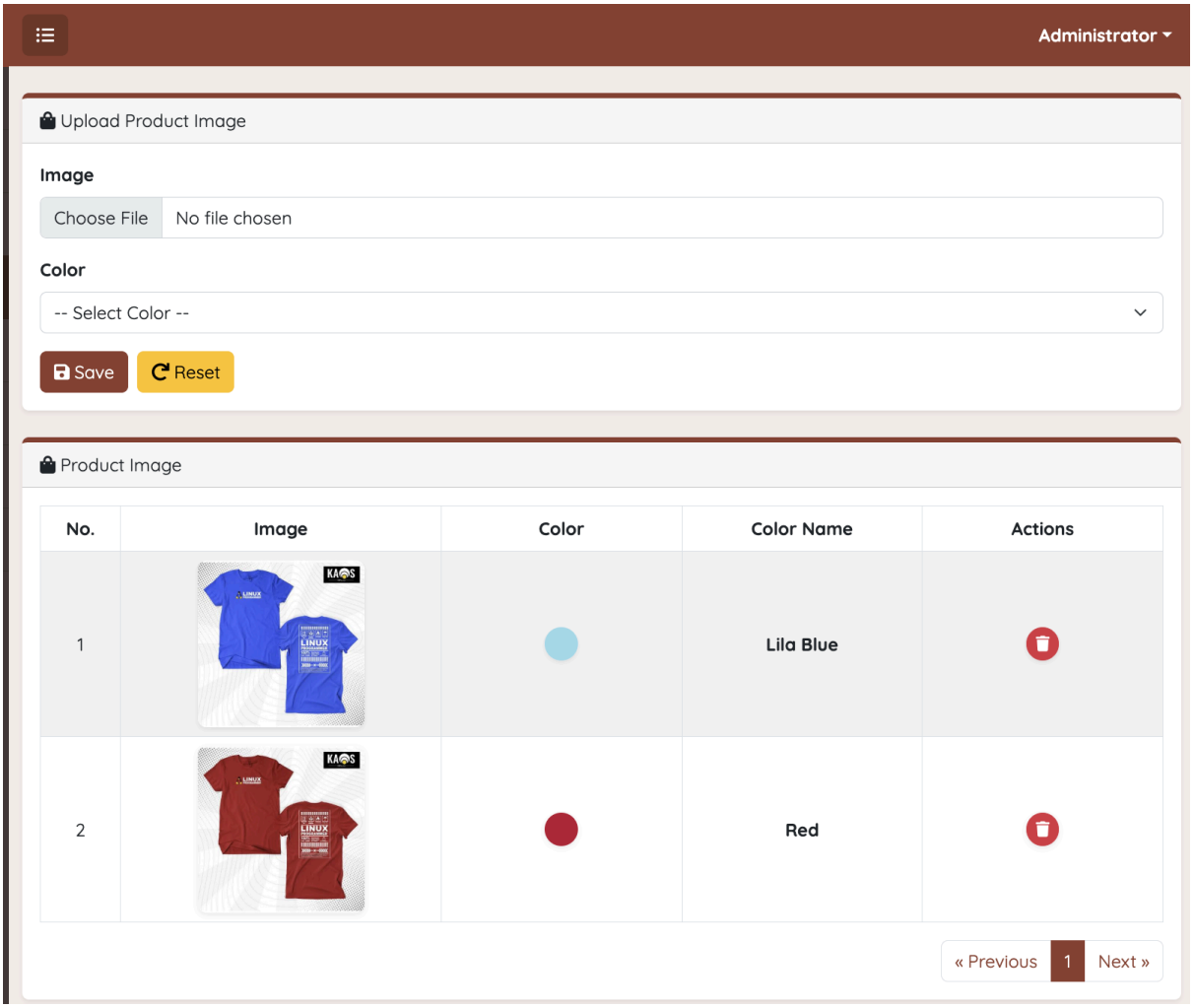


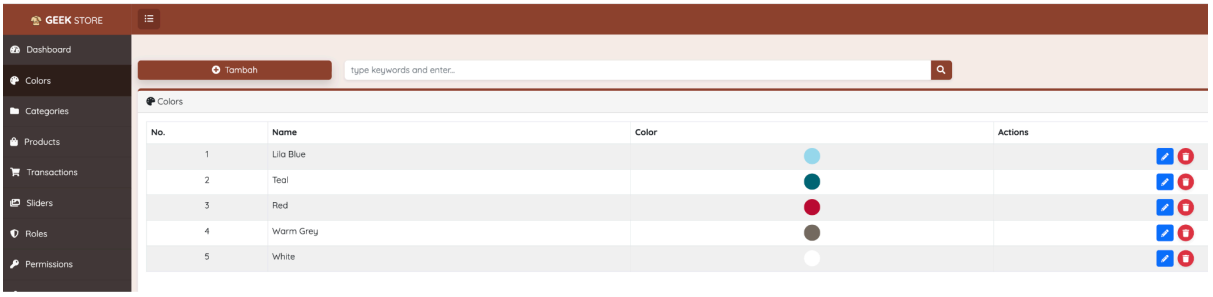
Masukan Gambar dan Warna



- **Colors Management**: Tambah dan atur varian warna produk.

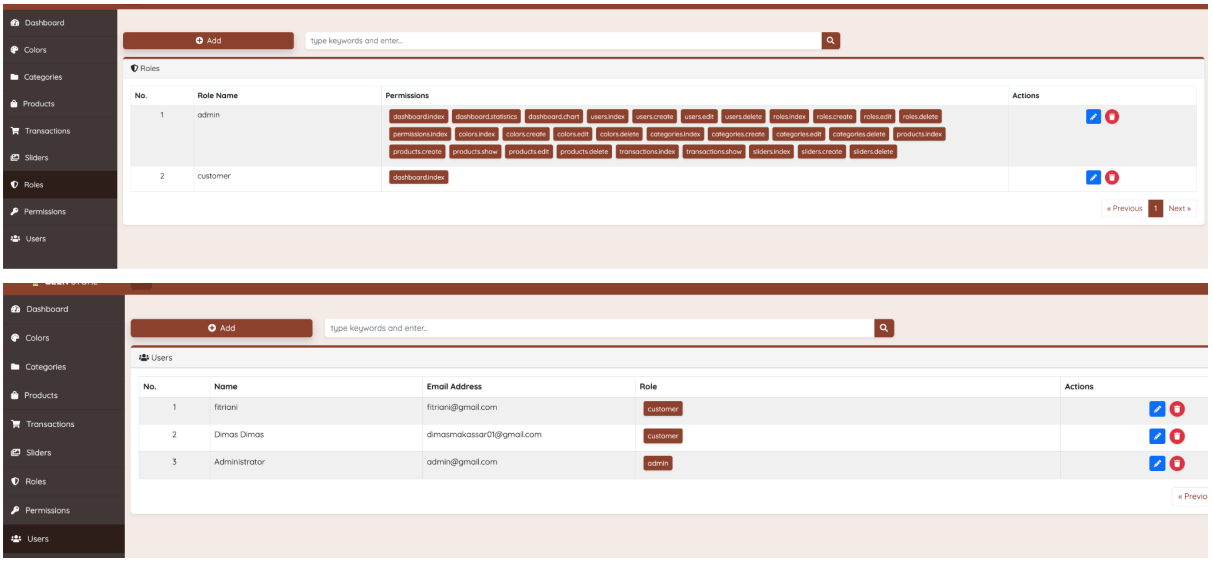






    - **Sliders Management**: Kelola gambar banner promosi di halaman utama toko.

- **Manajemen Pengguna & Hak Akses (User & Security Management)**:
    - **Users Management**: Kelola data pengguna/pelanggan dan penetapan role.
    - **Roles & Permissions Management**: Atur peran user dan hak akses fitur secara dinamis menggunakan Spatie Permission.



5.3 Alur Deployment ke GitHub

```
[ec2-user@ip-172-31-42-182 geek-store]$ git status
On branch main
Your branch is up to date with 'origin/main'.

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   README.md
        modified:   app/Http/Controllers/Account/TransactionController.php
        new file:   app/Http/Controllers/Account/TransactionController_31okt.php
        new file:
app/Http/Controllers/Account/TransactionController_ori_31okt.php
        modified:   app/Http/Controllers/Web/CheckoutController.php
        new file:   app/Http/Middleware/IsAdmin.php
        modified:   app/Models/Transaction.php
        new file:   app/Models/Transaction_31okt.php
        modified:   bootstrap/app.php
        new file:   bootstrap/app_ori.php
        modified:   bootstrap/cache/.gitignore
        modified:   resources/js/Pages/Account/Transactions/Index.jsx
        new file:   resources/js/Pages/Account/Transactions/Index_31okt.jsx
        modified:   resources/js/Pages/Account/Transactions/Show.jsx
        new file:   resources/js/Pages/Account/Transactions/Show_31okt.jsx
        modified:   resources/js/Pages/Web/Home/Index.jsx
        modified:   resources/js/Shared/CardProduct.jsx
        modified:   routes/web.php
        new file:   routes/web_dashboard_admin.php
        modified:   storage/app/.gitignore
        modified:   storage/app/private/.gitignore
        modified:   storage/app/public/.gitignore
        modified:   storage/framework/.gitignore
        modified:   storage/framework/cache/.gitignore
```

```
        modified:   storage/framework/cache/data/.gitignore
        modified:   storage/framework/sessions/.gitignore
        modified:   storage/framework/testing/.gitignore
        modified:   storage/framework/views/.gitignore
        modified:   storage/logs/.gitignore

[ec2-user@ip-172-31-42-182 geek-store]$ # 1. Tambahkan semua file yang baru diubah
git add .

# 2. Buat commit dengan pesan yang informatif
git commit -m "Fix: transaction details relationship, null safety, and image
rendering URL"
[main d6c4cff] Fix: transaction details relationship, null safety, and image
rendering URL

[ec2-user@ip-172-31-42-182 geek-store]$ git remote -v
origin  https://github.com/dimask016/project_uas_eligible.git (fetch)
origin  https://github.com/dimask016/project_uas_eligible.git (push)

[ec2-user@ip-172-31-42-182 geek-store]$ git remote add origin
https://github.com/dimask016/project_uas_eligible.git
error: remote origin already exists.

[ec2-user@ip-172-31-42-182 geek-store]$ # Cek nama branch aktif saat ini (biasanya
main atau master)
git branch -M main

# Push ke branch main
git push -u origin main
Username for 'https://github.com': dimask016
Password for 'https://dimask016@github.com':
Enumerating objects: 73, done.
Counting objects: 100% (73/73), done.
Delta compression using up to 2 threads
Compressing objects: 100% (36/36), done.
Writing objects: 100% (40/40), 12.37 KiB | 2.06 MiB/s, done.
Total 40 (delta 21), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (21/21), completed with 18 local objects.
To https://github.com/dimask016/project_uas_eligible.git
   18e10fc..d6c4cff  main -> main
branch 'main' set up to track 'origin/main'.
```